

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview	3
2. Background & Business Objective	4
3. Product Purpose	5
4. System Functionalities	5
5. User Stories & Use Cases	10
6. Features Included (Scope Definition)	14
7. Features Not Included (Scope Control)	14
8. Assumptions & Constraints	15
9. Risks & Questions Throughout Development	16

Project Name: Kira2 Je

Version: 1.0

1. Project Overview

- **Problem Statement:** Small and medium enterprises (SMEs) in the Food and Beverage industry may not have the capacity to hire a data analyst or set up a system to easily evaluate their costs and revenue. Many of these businesses already generate structured sales data through their POS systems, but their expense tracking remains analogue - most invoices are still physical documents requiring signatures for confirmation. This gap between digitised revenue and manual expense records means owners cannot easily see true cost vs revenue. Furthermore, there are no current solutions that cater to the small F&B enterprises in the Malaysian market specifically. Consequently, they are missing out on potential optimizations that could save costs and increase revenue.
- **Target Domain:** Enabling fast report generation for F&B SMEs for business optimization
- **Proposed Solution Summary:** Kira2 Je aims to help SMEs optimize their pricing schemes to maximize profits by analyzing historical menu item prices and costs to produce a comprehensive report with the recommended actions. It does this by combining POS sales exports with OCR-extracted invoice data, then using OpenAI's GPT-4o-mini as the analysis engine to surface specific actions with estimated monetary impact. The GPT-4o-mini model is central to the system - without it, the product is just a spreadsheet of raw numbers.

2. Background & Business Objective

- **Background of the Problem:** Most POS systems used by F&B SMEs offer limited analysis features, due to a lack of input channels for data such as item costs. To gain access to such features, the SMEs would need to invest in an enterprise resource planning (ERP) software or expensive data analytic services, which usually will cost an arm and a leg. Even though POS data is digitised, expense data still arrives as physical invoices, photos, and ad-hoc receipts - there is no easy way for an owner to bring both sides into a single view without manual data entry.
- **Importance of Solving This Issue:** The F&B industry adds to the Malaysian economy a staggering RM3.18 trillion, which is a result of a 91.5% increase since 2015. By removing the technical friction, any small business owner can analyze the business data, and receive professional-grade advice to enable them to make impactful business decisions, without needing a complex system or expensive advisor. Specifically, the target user is one who already runs on a POS system but has no time or tooling to properly reconcile expenses against revenue - not a street vendor, and not a large chain with its own analytics team.

Furthermore, the Malaysian government is pushing SMEs to digitalize their businesses by providing a RM5,000 grant under the MYDIGITAL initiative. Consequently, more and more businesses have adopted POS systems like StoreHub and Qashier as up to 30% of their business income now comes from online sales.

- **Strategic Fit / Impact:** Our main choice for this task would be OpenAI's GPT-4o-mini model, which provides excellent performance for minimum value. It does well with small focused tasks, and can generate reports with minimal cost.

3. Product Purpose

3.1. Main Goal of the System: To aid SMEs in Malaysia in decision making using an AI-powered financial advisor that sits on top of their existing business infrastructure to provide the insights that would normally require an ERP system.

3.2. Intended Users (Target Audience):

- Single-branch Cafe Owners
- Restaurant Owners
- Small F&B chains and catering businesses that already operate on a POS system

4. System Functionalities

4.1. Description: The solution acts as an intelligent decision maker for small and medium enterprises with an AI-powered engine at its core. The solution uses data extraction and AI vision to obtain the information it requires to perform a thorough analysis and provide contextual insights to aid decision making. The system bridges two data channels - digitised POS sales and analogue expense invoices - into a single AI-powered analysis engine, then returns recommendations with estimated monetary impact.

4.2. Key Functionalities:

- **Data Ingestion:** Receive and extract data from sources such as POS system exports and image/PDF uploads. Invoices can be submitted through three input methods that all feed the same OCR pipeline: camera scan for physical invoices, drag-and-drop / file upload for digital invoices and PDFs (with batch upload support), and a WhatsApp bot for zero-friction submission.
- **Record Generation:** Calculates gross revenue and gross profits, lists per-item profitability scoring, profitability drop alerts, historical comparison and menu item cannibalization detection and displays this information in a report. Each monthly report is generated in two views from a single API call and cached as, a "This Month" snapshot (top performers by RM profit, cost breakdown, at-risk items, ranked recommendations) and a "Trends" cumulative view (margin trajectory, supplier price changes, cannibalization alerts, month-over-month comparisons). The owner can toggle between the two views via a tab at the top of the report. Switching between views should not trigger another API call.
- **Confirmation Queue:** OCR-extracted invoices land in a pending queue where the owner can review, correct, or delete entries at their own pace. The owner can also return later to edit a previously confirmed invoice. If the invoice is edited, a new report is generated.

- **Report Export and Delivery:** Export generated reports to PDF, XLSX and DOCX formats, as well as sending the reports directly to a user's WhatsApp or email.
- **Dashboard:** Display data in graph form, as well as a list of past generated reports for the user to select, which will allow them to view data for the entire month or per-item. A persistent sidebar on the left lists reports chronologically with summary stats (revenue, margin) on a per month basis.
- **User Authentication and Onboarding:** No sign up page, users directly log in with their phone number, and first-time users are onboarded to determine business name, business type (restaurant/café/catering/etc.), POS system used and preferred language (BM or English). Optionally, the owner can connect WhatsApp during onboarding for invoice scanning.
- **What-If Simulator:** Accessible from any report. The owner can ask the GPT-4o-mini model hypothetical questions like "What if I raise teh tarik by RM0.50?" or "What if I drop this menu item?" and receive projected impact analysis using cumulative historical data.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: OpenAI's GPT-4o-mini model was selected as the optimal LLM to use. In a realistic application, GPT-4o-mini would be able to reduce the cost of each report generated while maintaining an affordable cost. Models such as Z.AI's GLM-5.1, Google's Gemini 2.5 and GPT-5.5 are powerful, but they are not as cost-efficient.

4.3.2. Prompting Strategy: Since AI models are known to hallucinate numbers to produce results, we decided to perform all calculations locally before sending them to the model. The role of the LLM model would be to analyze pre-computed data to make suggestions that aid users. The computational functions are stored within `lib/analytics.ts`.

All prompts that are sent to the GPT-4o-mini model have their own prompt builder which keeps them specialized for a single task and independent from each other. These prompts are stored in `lib/prompts.ts`, and are accessed when needed. The actual code that interacts with the OpenAI endpoint is stored in `lib/llm.ts`. Furthermore, based on the chosen language of the user, the `prompts.ts` script will send the system prompt in either Bahasa Malaysia or English. Each prompt specifies the JSON format that the model should return.

To ensure a standard format for each response, Zod schemas are used to validate the LLM's response. Zod is a declaration and validation library for TypeScript. These schemas are stored in `lib/schemas.ts`. All GPT-4o-mini responses are in JSON format, so they can be passed through schema checks.

For the chat feature of the solution, previously asked questions are saved and passed back to the model so it does not ask redundant questions and waste tokens.

Our prompting strategy is built around the principle where the LLM only handles analysis and language and everything it sends back is validated with schema guardrails. To keep the two-view report cheap, the report-generation prompt is designed to return both the "This Month" and "Trends" views in a single API call, with their structures defined together in the response schema.

4.3.3. Context & Input Handling: The primary feature of our solution is the report generation, which relies on the user's input. There are two primary input channels for the model's analyses, POS sales data via XLSX or CSV file upload and expense invoices via OCR (collected from in-app camera, drag-and-drop, or WhatsApp bot).

For the OCR channel, the user uploads images of their invoices, and this data is passed through Gemini's OCR model to extract meaningful data. In the case where it is unreachable, the solution will use Tesseract.js as a fallback. Details such as item name, quantity, cost per unit and total cost are extracted and stored in an array. This array is passed through along with the prompt from lib/prompts.ts. Extracted invoices land in the confirmation queue before being treated as analysis-ready data.

For the POS channel, our solution supports exported data from multiple POS systems that are popular in Malaysia. This channel does not require any AI, and data is directly passed to lib/analytics.ts for processing before it is passed to the model. If an owner uploads a multi-month XLSX dump, the system splits the data by month and generates a separate report for each.

When passing data to the model, the database is queried for all the associated user's menu items and their sales records. Records for the past one month and the month before that are retrieved. This context is provided to the model so that it can compare performance between months. Every metric is passed into the LLM via the reportPrompt in lib/prompts.ts, which instructs the model to produce a report that covers the required sections and cite actual item names and prices.

The what-if chat feature does not use all available metrics. It only takes a summary of the report, and the user's question is also passed verbatim. The model then goes through a predetermined chain of reasoning which is volume change, margin changes, cannibalization risk and estimated impact.

Lastly, whenever data is retrieved, the program checks for the user's ID, which is managed by lib/auth.ts. This ensures that a user will never see another user's data, ensuring data privacy.

4.3.4. Fallback & Failure Behavior: If any of the AI-related features encounter an error, it gracefully handles it and displays a pop up message informing the user that the service is temporarily unavailable. If the OCR feature fails, the program will use Tesseract.js as a fallback that runs locally. If only one data channel has been completed for the month (e.g., invoices are in but POS data is missing), the dashboard does not block, instead it shows what is missing and nudges the owner to complete the other channel.

5. User Stories & Use Cases

- **User Stories:**

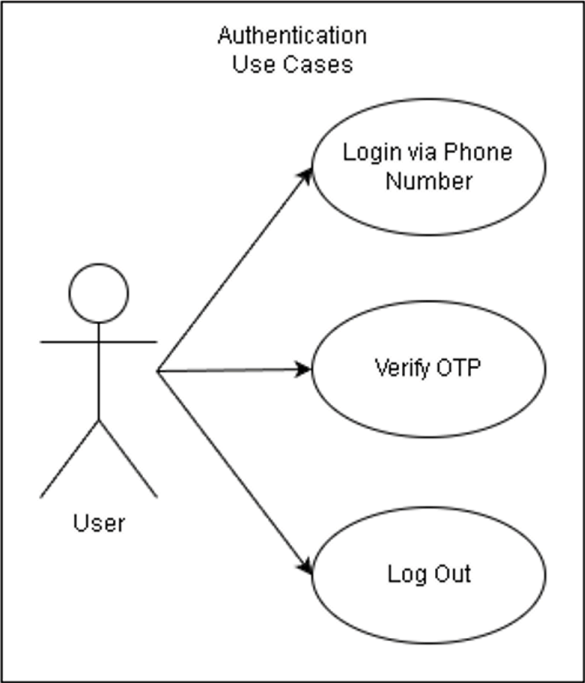
- “As a small business owner, I am unable to afford a complex ERP system for my business. Therefore, I would like a cheaper alternative that can handle data from multiple sources and run analysis on them.”
- “As a café owner who is always on the move, I want to snap a photo of an invoice the moment it arrives and forget about it - preferably without having to open yet another app.”
- “As an owner thinking about adjusting my menu prices, I want to ask the system what would happen if I raised an item by RM0.50 before I commit to the change.”

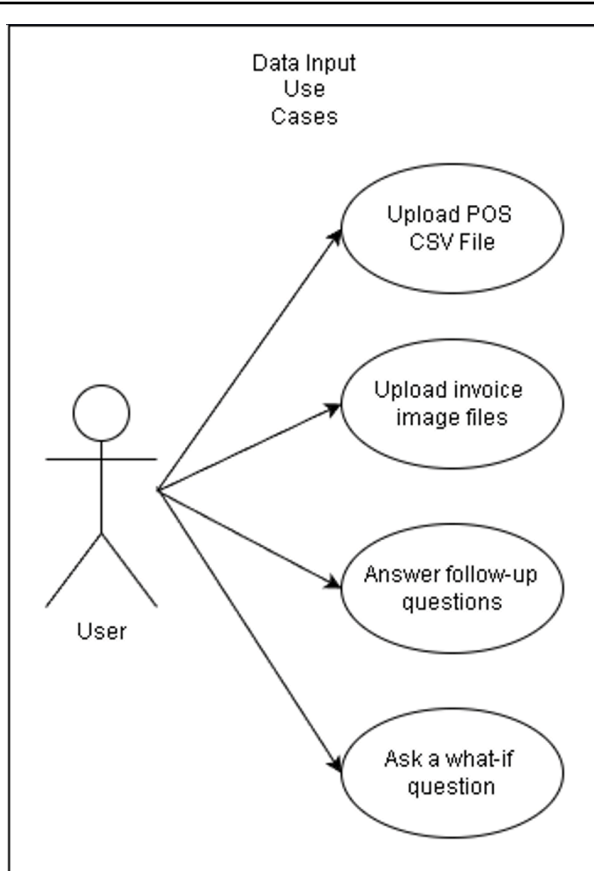
- **Use Cases (Main Interactions):**

- **CSV Data Upload to Report Generation:** Users upload sales data, then relevant data is extracted, extracted data is put into a standardized format to be parsed by GPT-4o-mini, produces analysis, and report is generated from results.
- **OCR Data Upload to Report Generation:** Users upload image of invoices, image is processed by OCR, extracted data is returned in JSON format, extracted data is reviewed in the confirmation queue, extracted data is put into a standardized format to be parsed by GPT-4o-mini, produces analysis, and report is generated from results.
- **WhatsApp Invoice Submission:** Owner sends an invoice photo to the Kira2 Je WhatsApp bot. The bot routes the image into the same OCR pipeline and the parsed result appears in the in-app confirmation queue, ready for review.

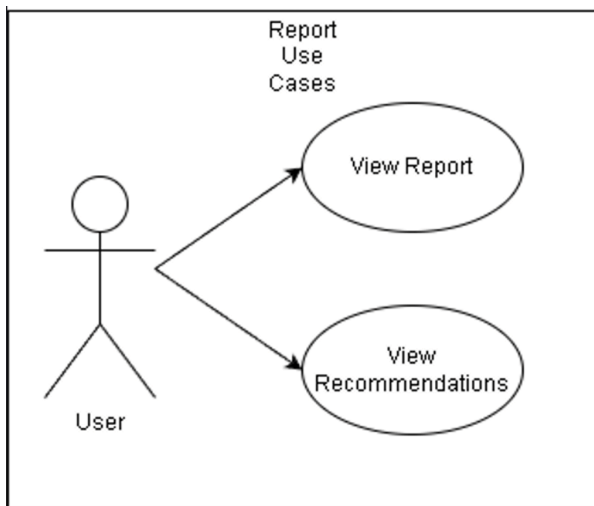
- **What-If Simulation:** From any report view, the owner asks a hypothetical pricing or menu question. The model reasons through projected volume change, margin change, cannibalization risk, and net monetary impact, then returns a structured response

- **Use Case Diagram:**

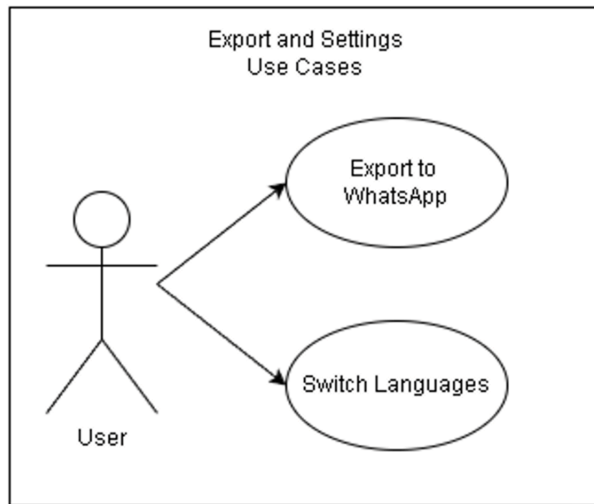
Use Cases	Description
 <pre> graph LR User((User)) --> Login(Login via Phone Number) User --> Verify(Verify OTP) User --> Logout(Log Out) </pre> The diagram shows a stick figure labeled 'User' on the left. Three arrows point from the user to three ovals on the right. The top oval is labeled 'Login via Phone Number', the middle one 'Verify OTP', and the bottom one 'Log Out'. The entire diagram is enclosed in a rectangular box with the title 'Authentication Use Cases' at the top center.	This use case diagram outlines the authentication-related use cases that the user can interact with.



This use case diagram outlines the data input use cases that the user can interact with.



This use case diagram outlines the report-related use cases that the user can interact with.



This use case diagram outlines the exports and settings-related use cases that the user can interact with.

6. Features Included (Scope Definition)

- AI-powered report generation
- AI-powered OCR data extraction
- AI-powered chat-style sales predictor
- Consistent design, flow and context throughout the whole system
- Support for Bahasa Malaysia and English
- Two-view monthly reports: single-month snapshot and cumulative trends, generated in a single API call
- Multi-channel invoice submission, in-app camera, drag-and-drop / file upload, and WhatsApp bot
- OCR confirmation queue with review, correction, and delete
- Persistent sidebar showing report history with per-month summary stats
- What-if simulator for pricing and menu-change projections
- Cannibalization detection

7. Features Not Included (Scope Control)

- Implementation of OTP being sent to real user mailboxes
- Export report to WhatsApp
- Tax calculator and Malaysian Tax System integration
- Inventory management
- Multi-branch / franchise features
- Payment processing - the system reads POS data, it does not replace the POS
- Full user authentication beyond basic phone-number login

8. Assumptions & Constraints

- **LLM Cost Constraint:** Per-business token usage is estimated at approximately 130,000–160,000 tokens per month under default settings (invoice OCR via vision model, report generation, and what-if questions). Routing OCR through a dedicated text-extraction library and only sending parsed text to the model for structuring can reduce this to roughly 50,000–70,000 tokens per business per month.
- **Technical Constraints:** Web-based React frontend, designed mobile-responsive so owners can scan invoices from a phone and review reports from a laptop. The persistent sidebar and report views adapt across viewport sizes. Native mobile apps and offline use are out of scope.
- **Performance Constraints:** Report generation requires both POS data and confirmed invoices for the target month. Therefore, until both channels have data, the dashboard shows partial state rather than an incomplete report. Cannibalization detection requires at least three months of historical data to produce reliable signals. Each monthly report is cached after generation so view-toggling does not re-bill API tokens.
- **User Input:** The system assumes the owner is already operating on a POS system that can export sales data as XLSX or CSV. OCR results require human confirmation in the queue before they are treated as analysis-ready, so the quality of the final analysis still depends on the owner reviewing parsed invoices.

9. Risks & Questions Throughout Development

- **Data Privacy:** How do we ensure users do not accidentally view other users' data?
- **AI Chat Stability:** How do we prevent or mitigate AI hallucinations?
- **Data Interpretation:** How will the error be handled if the AI is unable to interpret the data it is given?
- **Habit Formation:** Cannibalization detection only becomes useful after several months of consistent submissions. How do we keep owners submitting invoices regularly through the trough between month one and month three?
- **Mixed-Language and Handwritten Invoices:** Invoices in the wild come in BM, English, or a mix, and may contain handwriting. How do we communicate OCR confidence honestly to the owner so they know which extractions to double-check?
- **Edit-and-Regenerate Loop:** When an owner returns later to correct a previously confirmed invoice, the affected report must regenerate cleanly without confusing the user about which version they are looking at.